

Laboratorio de predicciones

El programa ya está escrito y ya funciona. Lo que se califica no es que corra: es **qué creías tú que iba a imprimir antes de correrlo**, y qué entendiste de las veces que te equivocaste. Es la práctica de tipos de datos y expresiones con operadores, hecha al revés.

En una línea

Abres `Laboratorio.java`, llenas la columna **PREDIJE** de las seis tablas **sin correr nada**, luego corres, llenas la columna **SALIO**, explicas cada fallo, y al final escribes tú las cinco líneas del bloque 7. Se entrega el `.java` con todo eso escrito, más el `.class`.

Por qué esta práctica es así

Copiar la salida de la pantalla no cuesta nada y no enseña nada. Predecir sí: en el momento en que escribes "7 / 2 da 3.5" y la máquina te contesta `3`, aprendiste algo que ya no se te va a olvidar en el examen ni en la Tarea 1. **Equivocarse aquí no baja la calificación**. Lo que la baja es no haber predicho, o no poder explicar el fallo.

Parte 0 · Qué se entrega y cómo se califica

QUÉ	DETALLE
Cuándo	Viernes 11 de septiembre , 23:59. Se empieza en el laboratorio del miércoles y ahí mismo se resuelven las dudas.

Dónde	Su rama <code>entregas_apellido_nombre</code> · carpeta <code>entregas/apellido_nombre/p03/</code> . Solo push. No se abre pull request.
Qué archivos	<code>Laboratorio.java</code> (con tus tablas llenas) y <code>Laboratorio.class</code> . Dos archivos, más nada.
Cómo se califica	Predicciones escritas antes de correr 30 % · explicación de tus fallos 30 % · las tres preguntas abiertas 20 % · bloque 7 corriendo 20 %.

Lo único que puede sacarte del 100

Que las 36 predicciones coincidan exactamente con la salida y no haya ni una explicación escrita. Eso no es una tarea perfecta: es la salida copiada. Nadie acierta las 36 la primera vez, y no se espera que lo hagan. **Ocho o diez fallos bien explicados es una entrega excelente.**

Parte 1 · El orden exacto de trabajo

1 Antes de correr nada: predice

Abre `Laboratorio.java` en tu editor. Cada bloque tiene una tabla en comentarios con dos columnas. Llena **solo la columna PREDIJE**, de los seis bloques, de corrido. Si no tienes idea, adivina algo: una predicción en blanco no vale, una equivocada sí.

2 Ahora sí, corre

`javac Laboratorio.java` y luego `java Laboratorio` . Sale todo de un jalón. Copia cada resultado a la columna **SALIO**. Aquí no hay que pensar, es transcribir.

3 Lo que de verdad cuenta: explica tus fallos

En cada bloque hay una línea **POR QUE ME EQUIVOQUE**. Escribe ahí, con tus palabras, por qué tu predicción no fue la que salió. Una línea por fallo. Si un bloque te

salió completo, escribe "0 fallos" y sigue. Contesta también las tres preguntas abiertas que están dentro de los bloques 3, 4, 5 y 6.

4 El bloque 7: ahora el código es tuyo

Cinco líneas comentadas que tienes que escribir y descomentar. Cada una con un comentario corto diciendo qué operador usaste y por qué. No vale escribir el resultado a mano: tiene que ser una operación de verdad que la máquina calcule.

Parte 2 · Qué hay en cada bloque

BLOQUE	DE QUÉ VA	RENGLONES
1	Con qué nace un atributo al que nadie le puso valor	5
2	La división que miente: entera contra decimal, y qué pasa al dividir entre cero	6
3	El módulo <code>%</code> , incluyendo con negativos y con decimales	6
4	El casting y lo que se pierde: decimales, letras y números que no caben	7
5	Relacionales y lógicos, y por qué una división entre cero a veces no truena	6
6	Las sorpresas: <code>0.1 + 0.2</code> , el entero que se da la vuelta, y el <code>+</code> que pega texto	6
7	Tuyo: cinco expresiones que tienes que escribir	5

Las que casi nadie acierta, para que no te asustes

Hay siete renglones diseñados para que falles: `(double) (7 / 2)`, `7.0 / 0`, `-7 % 2`, `(int) -3.9`, `(byte) 200`, `0.1 + 0.2 == 0.3`, y el par `1 + 2 + "3"` contra `"1" + 2 + 3`. Si fallas esos siete y los explicas bien, tu entrega está mejor que una donde nadie falló nada.

Parte 3 · Cómo se ve una respuesta que sí cuenta

El renglón `(int) -3.9`. Casi todos predicen `-4` y sale `-3`.

Explicación que no cuenta

"Me equivoqué porque pensé que era -4 pero es -3."

✓ Explicación que sí cuenta

"Pensé que el casting redondeaba, y no: corta la parte decimal y ya. Como corta hacia el cero, `-3.9` se queda en `-3`, no en `-4`. Si de verdad quisiera el `-4` tendría que usar otra cosa, no un casting."

La diferencia es que la segunda dice **qué creías que hacía el lenguaje y qué hace en realidad**. Ese es el ejercicio completo.

Parte 4 · Cómo se entrega

```
entregas/  
├─ tareas/  
│   └─ t1/           # la tarea del lunes, aparte  
└─ practicas/  
    └─ p3/  
        ├── Laboratorio.java  
        └─ Laboratorio.class
```

```
# 1. parate en TU rama (la misma de todo el semestre)  
git checkout entregas_apellido_nombre  
git pull  
  
# 2. la carpeta de esta practica  
mkdir -p entregas/apellido_nombre/p03
```

```
# 3. compila y corre una ultima vez antes de subir
javac Laboratorio.java
java Laboratorio
```

```
# 4. guarda y sube
git add entregas/apellido_nombre/p03
git commit -m "P3: laboratorio de predicciones"
git push -u origin entregas_apellido_nombre
```

✓ Checklist antes del último push

- ☐ Las 36 predicciones tienen algo escrito, ninguna en blanco
- ☐ Las 36 casillas de SALIO están llenas
- ☐ Cada bloque tiene su línea de POR QUE ME EQUIVOQUE, o dice "0 fallos"
- ☐ Contesté las preguntas abiertas de los bloques 3, 4, 5 y 6
- ☐ Las cinco líneas del bloque 7 están descomentadas, corren y tienen su comentario
- ☐ Escribí mi nombre hasta arriba del archivo
- ☐ `git branch` muestra el asterisco en mi rama, no en `main`
- ☐ Hice `push` y lo verifiqué abriendo mi rama en GitHub

Si algo falla

SÍNTOMA

```
class Laboratorio is public, should be
declared in a file named
Laboratorio.java
```

El programa corre pero el bloque 7 no imprime nada

```
Exception in thread "main"
java.lang.ArithmeticException: / by
zero
```

```
error: ';' expected en el bloque 7
```

QUÉ PASÓ

Le cambiaste el nombre al archivo. Este no se renombra: se queda

```
Laboratorio.java .
```

Sus cinco líneas siguen comentadas. Quítales el `//` de adelante.

En tu 7.5 usaste `&` en vez de `&&`, o pusiste la división del lado izquierdo. El corto circuito solo protege lo que está a la **derecha** de un `&&`.

Descomentaste la línea pero dejaste los tres puntos `...` adentro del paréntesis. Ahí va tu expresión.

`incompatible types: possible lossy
conversion from double to int`

Le estás dando un decimal a un `int` sin casting. Es justo lo del bloque 4.

`Error: Could not find or load main
class Laboratorio.class`

Corriste `java Laboratorio.class`. Es `java Laboratorio`, sin el `.class`.

Una cosa más

Esta práctica y la Tarea 1 son cosas distintas y van en carpetas distintas. La práctica es de aquí al viernes; la tarea es del lunes 14. Los siete renglones que fallaste en esta práctica son exactamente los errores que te van a salir en la tarea, así que hacerla primero te ahorra tiempo después.